

C++: THE TAD AWAKENS

DOMINE A FORÇA DOS TADS



Aprenda quais são os principais tipos de estruturas de dados da linguagem C++

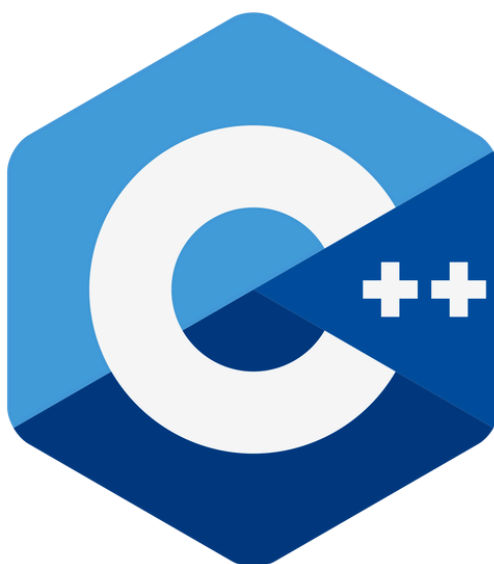
LEANDRO ANDRADE

Estruturas de Dados em C++

Guia Prático com STL

As estruturas de dados são ferramentas essenciais na programação. Elas organizam e armazenam informações de forma eficiente, permitindo manipular grandes quantidades de dados com rapidez. No C++, temos estruturas prontas na STL (Standard Template Library), que facilitam muito o nosso trabalho.

Neste eBook, você aprenderá as principais estruturas de dados do C++ usando suas bibliotecas nativas, com exemplos práticos e comentados.



01

VECTOR

O `std::vector` é uma versão moderna dos arrays tradicionais. Ele cresce automaticamente conforme novos elementos são adicionados.

VECTOR – A EVOLUÇÃO DOS ARRAYS



Um vetor (ou array) é uma sequência de elementos armazenados na memória, como uma linha de caixas numeradas onde cada número é um índice.

```
vector.cpp

1  #include <iostream>
2  #include <vector> // Biblioteca para vetores
3
4  using namespace std;
5
6  int main() {
7      vector<int> numeros = {10, 20, 30, 40, 50}; // Criamos um vetor
8
9      // Adicionamos um novo número
10     numeros.push_back(60);
11
12     // Percorremos e exibimos os elementos
13     for (int num : numeros) {
14         cout << num << " ";
15     }
16
17     cout << "\nTamanho do vetor: " << numeros.size() << endl;
18
19     return 0;
20 }
21
```



02

LISTAS ENCADEADA

A `std::list` é uma lista duplamente encadeada, o que significa que podemos inserir e remover elementos de qualquer posição com eficiência.

LISTA ENCADEADA - UMA CORRENTE FLEXÍVEL



Uma lista encadeada é como um trem, onde cada vagão (nó) sabe qual é o próximo. Isso nos permite adicionar ou remover elementos sem precisar reorganizar tudo.

```
list.cpp

1  #include <iostream>
2  #include <list>    // Biblioteca para listas
3
4  using namespace std;
5
6  int main() {
7      list<string> tarefas = {"Estudar C++", "Fazer exercícios", "Ler um livro"};
8
9      // Adicionamos uma nova tarefa no começo e no final
10     tarefas.push_front("Tomar café");
11     tarefas.push_back("Dormir");
12
13     // Removemos um elemento específico
14     tarefas.remove("Fazer exercícios");
15
16     // Exibimos a lista
17     for (const auto& tarefa : tarefas) {
18         cout << tarefa << endl;
19     }
20
21     return 0;
22 }
23
```



03

PILHAS

A `std::stack` segue a regra LIFO (Last In, First Out), ou seja, o último a entrar é o primeiro a sair.

PILHAS - ORGANIZAÇÃO EM CAMADAS



Imagine que você tem uma pilha de pratos, um em cima do outro. Quando você coloca um prato, ele fica no topo da pilha. E quando você precisa pegar um prato, você pega o que está no topo. Isso é uma pilha! Ela funciona assim:

- Último a entrar, primeiro a sair (LIFO – Last In, First Out).
- O último prato que você colocou na pilha é o primeiro que você vai pegar.
- Então, se você colocar 3 pratos, e depois pegar um, vai pegar o que está no topo, o último colocado.

```
stack.cpp

1  #include <iostream>
2  #include <stack> // Biblioteca para pilhas
3
4  using namespace std;
5
6  int main() {
7      stack<int> pilha;
8
9      // Empilhamos alguns números
10     pilha.push(10);
11     pilha.push(20);
12     pilha.push(30);
13
14     // Removemos os elementos do topo até a pilha ficar vazia
15     while (!pilha.empty()) {
16         cout << "Topo: " << pilha.top() << endl;
17         pilha.pop(); // Remove o elemento do topo
18     }
19
20     return 0;
21 }
22
```



04

FILAS

As filas seguem a regra FIFO (First In, First Out), como uma fila de banco. Podemos usar queue (fila simples) ou deque (fila de dois lados).

FILAS - ORDEM E PRIORIDADE



Imagine uma fila no banco. As pessoas entram na fila uma por uma, e a primeira pessoa que entra é a primeira a ser atendida. Isso é uma fila! Ela funciona assim:

- Primeiro a entrar, primeiro a sair (FIFO – First In, First Out).
- Quem chega primeiro na fila é atendido primeiro.
- Então, se 3 pessoas entram na fila, a primeira que chegou é a primeira que vai sair.

```
queue.cpp

1  #include <iostream>
2  #include <queue> // Biblioteca para filas
3
4  using namespace std;
5
6  int main() {
7      queue<string> fila;
8
9      // Adicionamos pessoas à fila
10     fila.push("Alice");
11     fila.push("Bob");
12     fila.push("Charlie");
13
14     // Atendemos as pessoas na ordem de chegada
15     while (!fila.empty()) {
16         cout << "Atendendo: " << fila.front() << endl;
17         fila.pop(); // Remove o primeiro da fila
18     }
19
20     return 0;
21 }
22
```



05

MAPAS

Os mapas são estruturas de chave-valor, como uma agenda telefônica.

FILAS - ORDEM E PRIORIDADE



Imagine que você tem uma agenda telefônica onde você guarda nomes e números de telefone. Para encontrar o número de alguém, você só precisa procurar o nome dessa pessoa. O mapa funciona de maneira bem parecida!

- Em um mapa, você tem chaves (como o nome na agenda) e valores (como o número de telefone associado a esse nome).
- A chave é única, assim como o nome de uma pessoa na agenda, e o valor é a informação que você quer associar àquela chave (como o número de telefone).

```
queue.cpp

1  #include <iostream>
2  #include <map>    // Biblioteca para mapas
3
4  using namespace std;
5
6  int main() {
7      // Criando um mapa onde a chave é o nome e o valor é o número de telefone
8      map<string, string> agenda;
9
10     // Adicionando pessoas e seus números de telefone ao mapa
11     agenda["Alice"] = "123-4567"; // Alice tem o número 123-4567
12     agenda["Bob"] = "987-6543";   // Bob tem o número 987-6543
13     agenda["Charlie"] = "555-6789"; // Charlie tem o número 555-6789
14
15     // Exibindo a agenda telefônica
16     for (const auto& entry : agenda) {
17         cout << "Nome: " << entry.first << " | Número: " << entry.second << endl;
18     }
19
20     // Consultando o número de telefone de uma pessoa (exemplo: Alice)
21     string nomeProcurado = "Alice";
22     cout << "O número de telefone de " << nomeProcurado << " é: " << agenda[nomeProcurado] << endl;
23
24     return 0;
25 }
26
```



AGRADECIMENTOS

OBRIGADO POR LER ATÉ AQUI



Esse Ebook foi gerado por IA, e diagramado por humano.
O passo a passo se encontra no meu Github

•

Esse conteúdo foi gerado com fins didáticos de construção,
foi realizado uma validação cuidadosa humana no conteúdo.



<https://github.com/leandroaa01/prompts-recipe-to-create-a-ebook>

Autor:

